Rotaract Club of Xiamen University Malaysia | Rotary Club of Kuala Lumpur Dynamics | Rotary Club of Tun Razak Exchange

# SDC Hackathon

# Technical Tools Declaration

**Hosted by Xiamen University Malaysia (XMUM) AI Club**
**Submission Deadline: 25 October 2026**

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

***Instagram:*** @sdc_xmumhack ***Email:*** sdc.xmumhack@gmail.com

**SDC HACKATHON 2026**

# Technical & Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: [Insert date]

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

## 1. Team & Project Overview

| | |
|---|---|
| **Team Name** | Hairloss |
| **Project / Solution Name** | SmartDesk |
| **Name (Leader)** | Judah Teoh Em Yu |
| **Email & Phone (Leader)** | teohjudah@gmail.com \| 0187765060 |
| **University Affiliation(s)** | Universiti Malaya |
| **Live Prototype URL** | https://orbyte-smartdesk.replit.app/ |
| **Code Repository URL (public / access granted to judges)** | https://github.com/nrehc9211/Smart-Desk |

## 2. Architecture & Technology Stack

### 2.1 System Architecture Overview

*Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.*

***[Maximum 200 words]***

A customer sends a WhatsApp message, which the WhatsApp Business Cloud API delivers to SmartDesk's webhook. The Node.js backend, deployed on Replit with API keys stored in Replit Secrets, saves the message in the SQLite database and calls the Google Gemini API to classify intent, extract order items and detect language. Routine questions receive template replies built from the owner's settings, and orders become draft invoices with eight automatic checks. Other questions go to the RAG module: Gemini embeddings search the FAQ knowledge base and live shop data, Gemini drafts an answer, and code verifies prices and sources. Drafts and invoices then appear in the owner dashboard, a web interface served by the same backend, where the owner approves, edits or rejects them. Approved replies are translated by Gemini when needed and sent back through the WhatsApp API, while approved invoices sync to Orbyte through its REST connector. A background scheduler sends payment reminders and follow-ups, and every step is written to the SQLite audit log, which feeds the dashboard and the hypothesis report. If Gemini is slow or unavailable, a built-in keyword engine keeps replies flowing, so the shop never stops.

### 2.2 Languages, Frameworks & Libraries

*List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.*

| Name & Version | Purpose in Project | License |
|---|---|---|
| JavaScript (ECMAScript 2022+) | Main programming language for the backend and dashboard | Open standard (ECMA-262), no license required |

| Name & Version | Purpose in Project | License |
|---|---|---|
| Node.js v24.x (minimum 22.5) | Server runtime; its built-in modules provide the web server, HTTP requests (fetch), database access (node:sqlite) and automated tests (node:test) | MIT License |
| HTML5 / CSS3 | Owner dashboard interface and Customer Simulator | Open web standards (W3C/WHATWG), no license required |
| Google Gemini API (gemini-3.8-flash, gemini-embedding-001) | Message understanding, RAG answer drafting, translation, hypothesis report; embeddings for meaning-based search | Proprietary cloud service, used under Google's Gemini API Terms of Service |
| Replit | Cloud development, Secrets storage and deployment of the prototyp | Proprietary platform, used under Replit's Terms of Service |
| Poppins and Noto fonts for pitch materials only | Typography and emoji in the pitch deck, flowchart and diagrams | SIL Open Font License 1.1 |

## 3. APIs & Cloud Services

### 3.1 External APIs

*List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.*

| API & Provider | Purpose in Project | Access Tier (free / paid / sponsor-provided) |
|---|---|---|
| **Orbyte REST API** (Orbyte) | Syncs owner-approved invoices to Orbyte | To be provided by Orbyte (connector built, currently running as a mock) |
| Gemini API | Reads customer messages (intent, items, quantity, language), drafts RAG answers from the FAQ, translates replies into any language, and groups unanswered questions into the hypothesis report | Google AI Studio API key |
| | | |
| | | |
| | | |

### 3.2 Cloud Platforms & Deployment

*List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.*

| Service & Provider | Purpose in Project | Plan / Region |
|---|---|---|
| Replit: Workspace and Deployment | Cloud hosting and compute: runs the Node.js server and owner dashboard, and gives the public web address used as the WhatsApp webhook | Paid plan |

| Service & Provider | Purpose in Project | Plan / Region |
|---|---|---|
| GitHub | Source code storage, version control, branches and pull requests for team collaboration; Replit imports the code from here | Free plan / global |
| Google AI Studio: Gemini API (Google) | Cloud AI compute for the LLM and embeddings (details in section 3.1) | Google-managed |
| | | |
| | | |

## 4. AI Tools, Models & Datasets

### 4.1 AI Models Used in the Solution

*List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.*

| Model & Version | Source / Provider | Usage (pre-trained / fine-tuned / self-trained) | License / Terms |
|---|---|---|---|
| gemini-3.8-flash | Google, via Gemini API (Google AI Studio) | **Pre-trained**, used through prompting: reads messages (intent, items, quantity, language, second questions), drafts RAG answers, translates replies, groups unanswered questions into hypotheses | Google Gemini API Terms of Service |
| **gemini-embedding-001** (embedding model) | Google, via Gemini API | **Pre-trained**: turns FAQ entries and customer questions into vectors for meaning-based RAG search, across languages | Google Gemini API Terms of Service |
| | | | |
| | | | |
| | | | |

### 4.2 Datasets

*List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.*

| **Dataset Name** | **Source / URL** | **Type (public / private / self-collected / synthetic)** | **License** | **Personal Data? (Y/N)** |
|---|---|---|---|---|
| FAQ knowledge base | knowledge/faq.json in our GitHub repo (github.com/nrehc9211/Hairloss) | Synthetic | Own work | N |
| Product catalogue and shop settings | Seed data in src/db.js | Synthetic | Own work | N |
| | | | | |
| | | | | |

### 4.3 AI-Assisted Development Tools

*Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.*

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☐ Yes ☐ No

If yes, list them below:

| **Tool & Version** | **Used For** | **Extent of Use (minor / moderate / major)** |
|---|---|---|
| Claude (Anthropic) | Writing most of the SmartDesk code (RAG module, Gemini integration, multilingual replies, translation safety check, time limits and fallbacks, automated tests); debugging; the pitch deck, flowchart and architecture diagram; the advert video; drafting written answers for this document | Major |
| **Replit Agent** (built into Replit) | Importing the project, configuring dependencies and the Run button, and checking the AI integration setup | Moderate |
| | | |
| | | |
| | | |

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

***[Maximum 100 words]***

Every AI-generated change was run and checked by our team before use. We ran the 42 automated tests after each update and tested real scenarios in the Customer Simulator with Malay, Chinese, English and Tamil messages. When results were wrong, such as replies in the wrong language, a missed halal question or slow AI responses, we reported them, the code was fixed and we retested. Changes went through GitHub branches and pull requests before merging. We checked pitch materials against the actual code. Our team understands the system and takes full responsibility for it.

## 5. Sponsor-Provided Resources

*Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.*

Did your project use any official SDC sponsor-provided resource?

☐ Yes ☐ No

If yes, list them below:

| **Sponsor** | **Resource (API / credits / platform / data)** | **How It Was Used in the Project** |
|---|---|---|
| Replit | Platform: cloud workspace and deployment | Imported our code from GitHub, ran the Node.js server in the cloud and published the prototype online. Its public web address serves the owner dashboard and acts as the WhatsApp webhook URL. |
| Replit | Platform feature: Replit Secrets | Stored the Gemini API key and WhatsApp credentials securely, so no secrets appear in our code or on GitHub |
| Replit | Tool: Replit Agent | Configured dependencies and the Run button during import, and helped check the AI integration setup |
| | | |

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

***[Maximum 100 words]***

Replit was essential to delivering our deployed prototype: it gave us a ready cloud environment without server setup, a stable public web address for the WhatsApp webhook, secure key storage through Secrets, and one-click publishing for testing and judging. However, we deliberately built SmartDesk as standard Node.js with no third-party dependencies, so the core functionality is not locked to one platform. This means the same

code also runs on a laptop or another cloud host, which lowers risk and makes it easier for other SMEs to adopt.

## 6. Security, Privacy & Responsible Use

### 6.1 Data Handling & Privacy

*What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.*

***[Maximum 150 words]***

SmartDesk collects customers' WhatsApp phone numbers and profile names, message text, and, when provided, names and delivery addresses, plus orders, invoices, payment status and an audit log of AI and owner actions. Data is stored in a SQLite database on Replit, excluded from GitHub, with API keys held in Replit Secrets. Third parties receive only what each task needs: Gemini receives message text, the shop's catalogue and FAQ, and replies for translation, which may include a customer's name or address, but never phone numbers; Meta's WhatsApp API delivers replies; Orbyte receives approved invoices once connected. Data is kept until the owner deletes it, with no automatic deletion yet. As Gemini's free tier may use inputs to improve Google's products, the prototype uses fictional demo data only. Phase 2 adds a paid tier, owner login, a fixed retention period and consent notices under Malaysia's PDPA 2010.

### 6.2 Known Limitations & Risks

*Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.*

***[Maximum 150 words]***

Hallucination: the LLM could invent facts, so it never sets prices; RAG answers use only retrieved FAQ entries, code flags drafts with unknown prices or sources for owner review, and anything outside the FAQ goes to the owner. Misreading messages: Manglish, mixed questions or uncommon languages may be misclassified, so low-confidence results escalate to the owner, owner-only topics like halal are always flagged, and 42 automated tests cover these cases. Translation errors: translated replies must keep every number and code, otherwise English is sent. Rate limits and outages: Gemini's free tier can return busy errors, so we use an 8-second limit, a retry, a backup model and a keyword fallback. Single-provider dependency: one setting switches to Claude. Other risks: WhatsApp follow-ups after 24 hours need approved templates, and owner login and webhook signature checks are planned for Phase 2.

## 7. Compliance & Readiness Checklist

*Tick all boxes that apply before submitting.*

- ☐ All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.
- ☐ All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.
- ☐ No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.
- ☐ Any use of generative-AI tools for development has been disclosed and reviewed by the team.
- ☐ Any use of official SDC sponsor resources has been declared in Section 5.
- ☐ The prototype described here is accessible to judges through the links provided in Section 1.
- ☐ All fields in this template have been completed within the word limits stated.

## 8. Declaration

By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team's knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.

[Signature]

[Date]